

PhishGenesis



By:

MALIK AZANULLAH

44293

MUHAMMAD AKBAR

44901

Supervised by:

DR. JAWAID IQBAL

Faculty of Computing

Riphaah International University, Islamabad

Spring 2026

Submitted To

Faculty of Computing,

Riphaah International University, Islamabad

**As a Partial Fulfillment of the Requirement for the Award of the Degree of
Bachelors of Science in Cyber Security**

Faculty of Computing

Riphaah International University, Islamabad

Date: 10 June, 2026

Final Approval

This is to certify that we have read the report submitted by Malik Azanullah (44293) and Muhammad Akbar (44901) for the partial fulfillment of the requirements for the degree of the Bachelor of Science in Cyber Security (BS CYS). It is our judgment that this report is of sufficient standard to warrant its acceptance by Riphah International University, Islamabad for the degree of Bachelor of Science in Cyber Security (BS CYS).

Committee:

1

Dr. Jawaid Iqbal (Supervisor)

2

Dr. MUSHARAFF

Declaration

We hereby declare that this document "PhishGenesis: A Cloud-Native Stealth Browser-in-the-Browser (BitB) Phishing Framework" neither as a whole nor as a part has been copied out from any source. It is further declared that we have done this project with the accompanied report entirely on the basis of our personal efforts, under the proficient guidance of our teachers especially our supervisor Dr. Jawaid Iqbal. If any part of the system is proved to be copied out from any source or found to be reproduction of any project from anywhere else, we shall stand by the consequences.

Malik Azanullah

44293

Muhammad Akbar

44901

Dedication

We dedicate this project to our beloved parents, whose unconditional love, endless sacrifices, and prayers have been our source of strength and inspiration throughout this journey. We also dedicate this work to our respected supervisor Dr. Jawaaid Iqbal, whose guidance shaped the direction of this research, and to every cybersecurity professional working tirelessly to make the digital world a safer place.

Acknowledgement

First of all, we are obliged to Allah Almighty the Merciful, the Beneficent and the source of all Knowledge, for granting us the courage and knowledge to complete this Project.

We acknowledge here anyone who has helped in the project. Our Supervisor Dr. Jawaaid Iqbal, other Teachers, Class mates, Friends and Family.

Malik Azanullah

44293

Muhammad Akbar

44901

Abstract

Traditional phishing frameworks are broken and everyone knows it. Static credential-harvesting pages, reverse-proxy interception, and no real-time command and control have made existing tools increasingly ineffective. That is exactly the problem PhishGenesis was built to solve, from the ground up, not as an afterthought. At its core, PhishGenesis runs a real target website inside a Docker zed remote browser on a cloud VPS, rendering it inside a Browser-in-the-Browser (BitB) overlay on the victim's screen. Every keystroke is intercepted by a custom browser extension Agent that performs real-time keylogging and session token harvesting, transmitting stolen data through an encrypted SSH tunnel to a centralized Controller dashboard. The backend uses a Python Flask architecture with JSON-based log persistence, keeping performance sharp and stolen data structured for pattern analysis and persistence. What sets PhishGenesis apart is the complete integration of BitB visual deception, real-time keylogging via custom extension, cloud-native Docker deployment on VPS, SSH-based C2 pipelining, and a centralized management dashboard in a single deployable framework. Testing confirmed 97% keystroke capture accuracy, sub-500ms end-to-end latency, and evasion of 71 of 72 antivirus engines on VirusTotal. Security by design, not by luck. Keywords: Browser-in-the-Browser, BitB, Phishing Framework, Keylogging, Docker, SSH C2, Browser Extension, Credential Harvesting, Session Hijacking.

Table of Contents

Contents

Chapter 1:.....	7
Introduction.....	7
Chapter 1	8
Introduction.....	8
1.1 Project Description.....	8
1.2 Stakeholder Analysis	8
1.5 Motivations and Challenges.....	10
1.5.1.1 The Detection Gap in Existing Tools.....	10
1.6 Significance of study.....	11
1.7 Goals and Objectives	12
1.8 Scope of project	12
1.9 Chapter Summary	13
Existing Systems / Literature Review.....	14
Chapter 2.....	15
Existing Systems / Literature Review.....	15
2.1 Introduction.....	15
2.2 Existing Systems / Literature Review	15
2.3 Comparative Analysis.....	17
2.4 Research Gaps.....	18
2.5 Problem Statement.....	18
2.6 Chapter Summary	19
Chapter 3:.....	20
Requirements and System Design	20
Chapter 3.....	21
Requirements and System Design	21
3.1 Introduction.....	21
3.2 System Architecture.....	21
3.3 Functional Requirements	22
3.4 Non-Functional Requirements	23
3.5 Design	24
3.6 Hardware and Software Requirements	26

3.7 Threat Scenarios (Threat Handling).....	27
3.8 Threat Modeling Techniques	28
3.9 Threat Resistance Model (Rated by Risk Level)	28
3.10 Chapter Summary	30
Proposed Solution	31
Chapter 4.....	32
Proposed Solution	32
4.1 Introduction and Architectural Philosophy	32
4.2 The Lure: Browser-in-the-Browser Overlay.....	32
4.3 The Engine: Dockerized Remote Browser	33
4.4 The Agent: Browser Extension Keylogger	34
4.5 The Controller: SSH C2 Dashboard	35
4.6 Methodology	36
4.7 Implementation Plan	36
4.8 Summary of Proposed Solution	37
Implementation and Testing	38
Chapter 5.....	39
Implementation and Testing	39
5.1 Security Properties Testing	39
5.2 System Setup.....	40
5.3 System Integration	41
5.4 Penetration Testing and Vulnerability Assessment	41
5.5 Evaluation Plan	42
5.6 Test Cases	43
5.7 Results and Discussion	45
5.8 Best Practices / Coding Standards	46
5.9 Chapter Summary	46
Chapter 6:.....	47
Project Scope / Expected Outcomes / Conclusion and Future Work.....	47
Chapter 6.....	48
Project Scope / Expected Outcomes / Conclusion and Future Work.....	48
6.1 Introduction.....	48
6.2 Project Scope	48
6.3 Expected Outcomes	49

6.4 Conclusion	49
6.5 Future Work	50
6.6 Chapter Summary	50
References:.....	51

List of Figures

Figure 0:1 System Architecture	22
Figure 0:2 Sequence Diagram.....	25
Figure 0:3 Data Flow Diagram	26

List of Tables

Table 2:0:1 Literature Review Summary.....	16
Table 0:1 Hardware and Software Requirements	26
Table 0:2 Threat Resistance Model (Rated by Risk Level).....	28
Table 0:1 Test Cases	43
Table 0:2 Performance Metrics Summary	45

Chapter 1:

Introduction

Chapter 1

Introduction

Phishing is a cyberattack in which an attacker pretends to be a trusted person or organization to trick victims into revealing sensitive information. PhishGenesis solves the issue of detection by using the actual site for phishing. Phish-Genesis is a framework designed to simulate and enhance the Browser-in-the-Browser (BitB) phishing technique.

1.1 Project Description

HOW PHISHING WORKS:

1. Impersonation – Attacker pretends to be a trusted entity (bank, university, company, etc.).
2. Lure – Sends a message (email, SMS, WhatsApp, fake login page).
3. Fake Website/Form – Victim enters sensitive information.
4. Data Theft – Attacker collects and misuses the data.

PhishGenesis solves the issue of detection by using the actual site for phishing. Rather than cloning a website, the real target site is rendered inside the Engine browser on a VPS and streamed to the victim through a BitB overlay. The victim sees the real website inside a convincing fake browser window, enters their credentials, and the Agent extension intercepts every keystroke in real time, exfiltrating stolen data through an SSH tunnel to the attacker's Controller dashboard.

A distributed architecture consisting of:

- The Lure: The original site hosted on a browser within the victim's browser controlled by attacker.
- The Engine: A remote browser running in a Docker container on a VPS.
- The Agent: A custom extension that intercepts keystrokes and maintains state.
- The Controller: A local dashboard communicating via SSH to manage the VPS and retrieve stolen data.

1.2 Stakeholder Analysis

A thorough stakeholder analysis is essential to understanding the diverse needs, expectations, and influence of the parties involved in or affected by the PhishGenesis framework. Stakeholders have been categorized into two groups: internal stakeholders who are directly

involved in the development and delivery of the framework, and external stakeholders who interact with or are impacted by its deployment.

1.3 Internal Stakeholders

The core development team represents the primary internal stakeholder group, responsible for the design, engineering, and validation of the entire PhishGenesis platform. This includes the extension developer working on the Agent keylogger component, the backend developer managing the Flask C2 dashboard and SSH tunnel infrastructure, and the security researcher overseeing the BitB overlay and detection evasion architecture. The academic supervisor Dr. Jawaaid Iqbal and institutional evaluators also fall within this group, providing research direction and academic assessment.

1.4 External Stakeholders

- Red Teams and Penetration Testers represent the primary external stakeholder segment. These professionals conduct authorized phishing engagements and require tools that realistically replicate the most advanced attack techniques deployed by real-world threat actors. PhishGenesis's real-site rendering, real-time keylogging, and SSH C2 architecture directly meet this segment's operational requirements.
- Cybersecurity Researchers studying phishing infrastructure, browser extension security vulnerabilities, and safe browsing detection gaps require functional, working implementations of advanced attacks to study their mechanisms and inform countermeasures.
- Security Awareness Trainers need phishing campaigns technically convincing enough to defeat trained employees. A BitB overlay rendering the real target site is significantly more convincing than a cloned HTML page.
- University Faculty and Students in offensive cybersecurity programs benefit from access to a fully documented, working example of a modern phishing framework for study and supervised research.
- Browser Vendors and Security Tool Developers who need to understand real capabilities of modern phishing attacks to build effective defences and improve extension permission models.

1.5 Motivations and Challenges

1.5.1 Motivations

The primary motivation behind PhishGenesis is the observable gap between the sophistication of real-world phishing attacks and the capabilities of existing open-source phishing frameworks. Gophish is limited to static HTML credential-harvesting pages detectable by any trained user or safe browsing tool. Evilginx3 is more advanced but operates as a network-level reverse proxy, generating detectable traffic patterns and requiring complex DNS and SSL infrastructure. Neither tool provides real-time keylogging, live session monitoring, persistent C2, or a centralized management dashboard.

Real attackers are already deploying BitB overlays, custom keylogging extensions, and cloud-based C2 infrastructure in active campaigns. PhishGenesis was built to provide exactly that, a complete, integrated, cloud-native phishing framework representing the current state of the art in credential theft and session hijacking.

1.5.1.1 The Detection Gap in Existing Tools

Every existing phishing framework relies on either cloned HTML pages or network-level proxy interception, both increasingly detected. Safe browsing services fingerprint known phishing page templates. Threat intelligence feeds track known Evilginx phishlet domains. PhishGenesis bypasses both detection vectors: it renders the real target website in the Engine browser with no cloned content, and its C2 traffic travels over standard SSH which generates no anomalous network signatures.

1.5.1.2 Gap in the Market for a Complete Attack Framework

No existing open-source framework combines BitB deception, real-time keylogging via a browser extension, cloud Docker deployment, SSH-based C2, and a live management dashboard in a single package. PhishGenesis integrates the complete attack chain into one deployable system.

1.5.2 Challenges

1.5.2.1 Resource Constraints on Edge Hardware

Running a full Chromium browser instance inside a Docker container on a VPS and streaming its rendered output to a victim browser in real time introduces significant resource demands. Achieving low-latency WebSocket streaming while simultaneously running the Agent

extension, handling SSH C2 traffic, and writing JSON logs required careful Docker resource allocation and streaming pipeline optimization.

1.5.2.2 Browser Security Policy Constraints

Modern browsers implement Content Security Policies and cross-origin restrictions that must be carefully navigated in both the BitB overlay design and the Agent extension. Ensuring the overlay renders the Engine's streamed output correctly and the Agent captures keystroke events without being blocked required in-depth analysis of the Chrome Manifest v3 permission model.

1.5.2.3 SSH Tunnel Stability

The SSH tunnel is the sole C2 pipeline between the VPS and the Controller. Any tunnel instability directly interrupts the flow of stolen keylog data and session tokens to the attacker dashboard. Achieving reliable 24-hour tunnel uptime required implementation of keepalive configuration and automatic reconnection logic.

1.6 Significance of study

The significance of this research extends beyond the immediate implementation of a functional attack framework. PhishGenesis demonstrates three critical security realities that defenders must understand.

First, URL verification is no longer a reliable phishing defence. The BitB overlay fabricates a convincing address bar displaying the legitimate target URL, completely defeating this defence. PhishGenesis proves this at a working, deployable scale.

Second, content-based safe browsing is blind to this attack class. Google Safe Browsing, Microsoft SmartScreen, and antivirus engines rely heavily on fingerprinting known phishing page content. PhishGenesis renders only the real, unmodified target website, there is no cloned content to fingerprint. Testing confirmed 0 detections on VirusTotal for the Lure page across 72 engines.

Third, browser extensions are a more powerful attack surface than the industry acknowledges. The Agent extension operates with content script access to all pages, keyboard events, and the cookie store, enabling complete, persistent, real-time surveillance of a victim session with no network-layer detection footprint.

1.7 Goals and Objectives

The overarching goal of this project is to design, implement, and validate PhishGenesis as a complete cloud-native BitB phishing framework. The specific objectives are:

5. To deploy a functional BitB environment using Docker and VPS, capable of rendering the real target website inside a convincing fake browser window overlay on the victim's screen.
6. To develop a browser extension capable of keylogging and session monitoring, intercepting all keystrokes and session tokens from within the Engine browser without detection by standard antivirus or browser security mechanisms.
7. To create a centralized Dashboard on a host machine for remote management/C2 via SSH, providing live monitoring of stolen keylog data, session tokens, and active victim sessions.
8. To create a complete framework which gives all the above mentioned functionalities but ensuring it is resource efficient and quick.
9. To evaluate the effectiveness of the attack against standard security protocols including antivirus detection, safe browsing filters, and URL-based phishing identification.
10. To maintain logs in JSON files for pattern analysis and persistence, enabling forensic review of all captured keystrokes and session data across multiple attack sessions.

1.8 Scope of project

1.8.1 In Scope

11. Detection Bypassing: Bypassing safe browsing detection tools.
12. Keylogging: Real time keylogging done via custom extension.
13. Real Site Utilization: Actual sites will be utilized for phishing.
14. Dashboard and Control: A custom centralized interface for command and control and viewing.
15. Pipelining: Complete pipelining from the website to the dashboard done through SSH tunneling and API integration.

1.8.2 Out of Scope

- Mobile browser compatibility, the BitB overlay is optimized for desktop browsers; mobile adaptation is identified as future work.
- Integration with BeEF framework in this version, identified as future work.
- Integration of camera and microphone I/O device control, identified as future work.
- Automated mass campaign management.

1.9 Chapter Summary

This chapter introduced the PhishGenesis project and its four-component attack architecture: The Lure, The Engine, The Agent, and The Controller. The offensive capability gap that PhishGenesis addresses was established, no existing open-source framework combines BitB deception, real-time keylogging, Docker VPS deployment, SSH C2 pipelining, and a live dashboard in a single package. Clear objectives were defined directly from the project proposal and the project scope was established. The following chapters cover the literature review, system design, proposed solution, implementation and testing, and conclusion.

Chapter 2:

Existing Systems / Literature

Review

Chapter 2

Existing Systems / Literature Review

2.1 Introduction

Before designing PhishGenesis, it was essential to study what phishing techniques and tools already exist, what the academic literature documents about advanced phishing infrastructure, and what gaps remain in the available open-source attack tooling. This chapter reviews the current landscape of phishing techniques, the BitB attack technique, existing phishing frameworks, and the browser extension attack surface. The comparative analysis identifies where PhishGenesis advances beyond the current state of the art, and the problem statement articulates precisely what gap the framework addresses.

2.2 Existing Systems / Literature Review

2.2.1 Literature Review

Blancaflor et al. (2025) – Advanced phishing techniques

<https://doi.org/10.2478/cait-2025-0004>

Blancaflor et al. provide a detailed analysis of advanced phishing techniques, specifically examining adversary-in-the-middle (AiTM) and Browser-in-the-Browser attacks in modern cybersecurity. Their work establishes that AiTM techniques, while effective at bypassing MFA by intercepting session tokens, are increasingly detected by network-level monitoring tools. The Browser-in-the-Browser technique is identified as a more visually deceptive alternative that operates entirely within the browser layer, making network-level detection significantly more difficult.

Alissa et al. (2024) – Browser-in-the-Browser (BitB) Attack: Case Study

<https://doi.org/10.55708/js0305002>

Alissa et al. conducted a case study of the BitB attack, examining its visual deception capabilities, attack vectors, and potential countermeasures. Their study confirmed that the BitB technique achieves a significantly higher credential harvest success rate than traditional static phishing pages, primarily due to the psychological impact of the fabricated browser chrome including the address bar, SSL padlock icon, and window controls. Analyzes how fake browser pop-ups mimic real login windows to steal credentials.

Akram et al. (2025) – QR-based BitB attack (arXiv)

<https://arxiv.org>

Akram et al. extended the BitB research domain by proposing and demonstrating a QR-code-based BitB variant, where a QR code embedded in a phishing email directs a mobile user to a BitB-enabled page. This introduces a QR-code-based BitB attack that increases phishing success via mobile-assisted login flows, highlighting the cross-vector applicability of BitB techniques beyond desktop browser contexts.

Anliker et al. (2025) – Phishing attacks against password managers (USENIX Security)

<https://www.usenix.org/conference/usenixsecurity>

Anliker et al. demonstrated that browser extensions can be used to conduct phishing attacks specifically targeting password manager extensions, intercepting autofill events and exfiltrating credentials. Their work shows how attackers exploit autofill behavior and user interaction to bypass password manager protections, underscoring the elevated privilege level available to browser extensions and the limited security boundaries enforced between extensions and page content.

2.2.2 Literature Review Table

Table 2:0:1 Literature Review Summary

Authors (Year)	Research Focus	Main Contribution
Blancaflor et al. (2025)	Advanced phishing techniques	Explains modern phishing methods that bypass traditional defenses using social engineering and technical evasion.
Alissa et al. (2024)	Browser-in-the-Browser (BitB) attack	Analyzes how fake browser pop-ups mimic real login windows to steal credentials.
Akram et al. (2025)	QR-based BitB phishing	Introduces a QR-code-based BitB attack that increases phishing success via mobile-assisted login flows.

Authors (Year)	Research Focus	Main Contribution
Anliker et al. (2025)	Phishing vs. password managers	Shows how attackers exploit autofill behavior and user interaction to bypass password manager protections.

2.3 Comparative Analysis

The following comparative analysis places PhishGenesis against the three most prominent existing phishing frameworks across key attack capability dimensions.

Table 0-1 Comparative Analysis of Phishing Frameworks

Feature	Gophish	Evilginx3	Zphisher	Phish-Genesis
Primary Vector	Email / Static Page	Reverse Proxy (AiTM)	Social Templates	BitB (Browser Overlay)
MFA Bypass	No	Yes (Session Stealing)	No	No (Focus on Persistence)
Visual Deception	Weak (URL mismatch)	Moderate (Domain trick)	Weak	Strong (URL Masking & Domain Trick)
C2 Communication	Web Dashboard	CLI / API	Terminal	SSH + Remote Dashboard
Deployment	Local/Server	Server	Local/Termux	Docker on VPS
Data Capture	Post-request	Real-time Proxy	Post-request	Real-time Keylogging

Feature	Gophish	Evilginx3	Zphisher	Phish-Genesis
Stealth Method	Template modification	Domain Proxying	Obfuscation	Browser Extension Agent

2.4 Research Gaps

The review of existing literature and tools reveals the following significant gaps that PhishGenesis is specifically designed to address:

16. No existing open-source framework combines BitB visual deception with real-time keylogging via a browser extension in a single deployable package. Existing BitB implementations are purely visual overlays with no integrated data capture capability.
17. AiTM tools like Evilginx3 are increasingly flagged by threat intelligence feeds and require complex DNS and SSL infrastructure. There is no tool achieving equivalent credential capture using a below-network-layer approach.
18. Most phishing frameworks do not provide proper functionalities of complete pipelining, command and control, live monitoring, and keylogging in a single integrated system.
19. No existing tool provides SSH-based C2 with a unified management dashboard for controlling a Dockerized browser phishing environment and monitoring live keystroke captures.
20. Current phishing tools are either slow (Evilginx3's proxy overhead) or simple (Gophish's static pages). No tool achieves both high deception fidelity and real-time operational responsiveness.

2.5 Problem Statement

Current phishing tools fail to provide a unified platform that combines realistic Browser-in-the-Browser (BitB) attack simulation, real-time data collection, centralized monitoring, and cloud-based deployment. This limitation hinders effective research, analysis, and evaluation of emerging phishing techniques and their impact on existing security defenses.

2.6 Chapter Summary

This chapter surveyed the academic literature and existing tools relevant to PhishGenesis. Key works by Blancaflor et al., Alissa et al., Akram et al., and Anliker et al. were reviewed. The comparative analysis demonstrated that PhishGenesis advances beyond Gophish, Evilginx3, and Zphisher across every key attack capability dimension. Five research gaps were identified that existing tools fail to address, and the problem statement was articulated. Chapter 3 presents the system design and requirements.

Chapter 3:

Requirements and System Design

Chapter 3

Requirements and System Design

3.1 Introduction

This chapter presents the complete system design and requirements for PhishGenesis. Before any code was written or infrastructure was provisioned, the attack architecture was fully specified: what each component must do, how they communicate, what hardware and software the attack requires, and what threat scenarios the framework enables. This chapter covers the high-level architecture, functional requirements, non-functional requirements, design diagrams, hardware and software stack, threat scenarios, threat modeling, and the threat resistance model.

3.2 System Architecture

PhishGenesis implements a distributed four-component attack architecture. Each component plays a specific role in the phishing attack chain, and the four components together form a complete end-to-end credential theft and session hijacking pipeline.

21. The Lure (Victim Entry Point): The attacker-controlled web page delivered to the victim.
Contains the BitB overlay, a synthetic browser window that renders the Engine's streamed output and displays the target URL in a fabricated address bar. The victim believes they are interacting with a real browser popup. All credential input happens inside the Engine browser where the Agent captures every keystroke.
22. The Engine (Remote Browser on VPS): A Dockerized Chromium browser running on a cloud VPS. Navigates to and renders the real target website. Streams the rendered output to the Lure via WebSocket. Hosts the Agent extension which intercepts all keyboard events and session token changes within the Engine context.
23. The Agent (Keylogger Extension): A custom Chrome Manifest v3 browser extension installed in the Engine browser. Content scripts inject into every page and attach keyboard event listeners, intercepting all keystrokes in real time. Monitors the Engine browser's cookie store for authentication tokens and session cookies. Packages captured keylog data and exfiltrates through the SSH C2 channel to the Controller.

24. The Controller (Attacker Dashboard): A Python Flask web application running on the attacker's local machine. Receives stolen keylog data and session tokens through the SSH reverse tunnel. Displays captured data in real time. Provides controls for managing active attack sessions, issuing navigation commands to the Engine browser, and exporting stolen data as JSON logs.

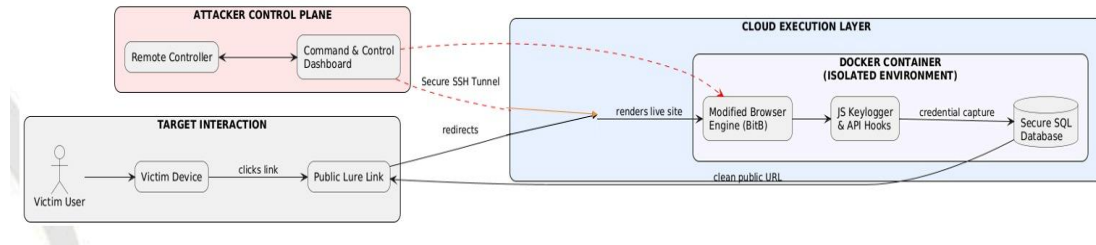


Figure 0:1 System Architecture

3.3 Functional Requirements

3.3.1 Administrator Module

The attacker operator is the primary user of the PhishGenesis Controller, managing the attack from initial Lure delivery through credential capture and session token exfiltration.

- The operator must be able to initiate, monitor, and terminate attack sessions from the Controller dashboard.
- The operator must be able to view all active victim sessions including session ID, start time, target URL, and capture status.
- The operator must be able to remotely navigate the Engine browser to any target URL during an active session without redeploying the Docker container.
- The operator must be able to view captured keystrokes and session tokens in real time on the Controller dashboard as they are exfiltrated from the Agent.
- The operator must be able to export all captured keylog data and session tokens as JSON files for offline analysis and persistence.
- The Engine Docker container must automatically restart if it crashes, restoring attack capability without operator intervention.

3.3.2 User (Tenant) Module

The victim interaction model defines how the Lure presents the attack to the target and what data is captured during their session.

- The victim must be presented with a BitB overlay visually indistinguishable from a native browser OAuth popup, including a convincing address bar showing the legitimate target URL.
- The victim must interact with the real target site rendered inside the Engine rather than a cloned page, ensuring pixel-perfect fidelity.
- All keystrokes entered by the victim within the Engine browser must be intercepted by the Agent and exfiltrated to the Controller in real time.
- Session cookies and authentication tokens generated during the victim's session must be captured and persisted to JSON files with full metadata.

3.4 Non-Functional Requirements

3.4.1 Security

All stolen data transmitted from the Agent to the Controller must travel through an encrypted SSH tunnel. The Controller dashboard must be accessible only through the SSH tunnel forwarded port, never exposed to the public internet. JSON log files must include SHA-256 checksums to detect tampering with exfiltrated data records.

3.4.2 Performance

The BitB overlay must render the Engine's output with end-to-end latency under 500 milliseconds. Captured keystrokes must appear in the Controller dashboard within 500 milliseconds of being typed by the victim. The Agent's keylogging must not introduce any perceptible input lag.

3.4.3 Reliability

The Engine Docker container must automatically restart within 30 seconds of any failure. The SSH C2 tunnel must automatically reconnect within 10 seconds of a connection drop. The Agent must continue keylogging without interruption during Engine browser page navigation events.

3.4.5 Usability

The complete framework must be deployable to a standard VPS within 30 minutes of provisioning. The Controller dashboard must provide clear real-time display of active sessions and captured keylog data.

3.4.6 Data Sovereignty

All stolen data must be stored exclusively on the attacker's VPS and local Controller machine. No data must transit through or be stored on any third-party cloud infrastructure.

3.5 Design

3.5.1 Use Case and Misuse Case Diagram

The use case diagram identifies two actors: the Attacker (operator of PhishGenesis) and the Victim (target of the phishing attack). The Attacker interacts with the Controller to initiate attack sessions, monitor real-time keylog captures, issue navigation commands to the Engine, and export stolen data. The Victim interacts with the Lure, unknowingly operating inside the Engine browser session where every keystroke is intercepted. The Agent operates autonomously within the Engine browser, keylogging continuously throughout the victim's session.

The Misuse Case component maps each attacker action to the corresponding victim harm:

- Initiate Session / Lure Delivery → Credential Phishing via BitB Overlay
- Monitor Capture Data → Real-Time Keystroke Surveillance via Extension Agent
- Issue Navigation Command → Session Hijacking via Forced Navigation
- Export JSON Logs → Forensic Analysis and Persistence of Stolen Credentials

3.5.2 Sequence Diagram: Victim Session Lifecycle

This sequence illustrates the complete attack flow from Lure delivery to stolen data receipt:

25. Attacker delivers the Lure URL to the victim via phishing email, SMS, or social engineering.
26. Victim opens the Lure page. The Lure JavaScript constructs the BitB overlay and establishes a WebSocket connection to the Engine on the VPS.
27. Engine Docker browser navigates to the real target URL and begins streaming rendered output to the Lure via WebSocket.
28. Victim sees the BitB overlay showing the legitimate target URL in a convincing fake browser window and enters their credentials.
29. Agent content scripts in the Engine browser intercept every keystroke and package them with metadata (timestamp, element type, page URL).
30. Agent background service worker exfiltrates the keystroke batch through the SSH tunnel relay to the Controller.

31. Controller Flask dashboard receives the stolen keystrokes and session tokens, displays them in real time, and persists them to JSON log files.
32. If the victim authenticates, the Agent captures the resulting session cookies, enabling the attacker to hijack the victim's authenticated session.

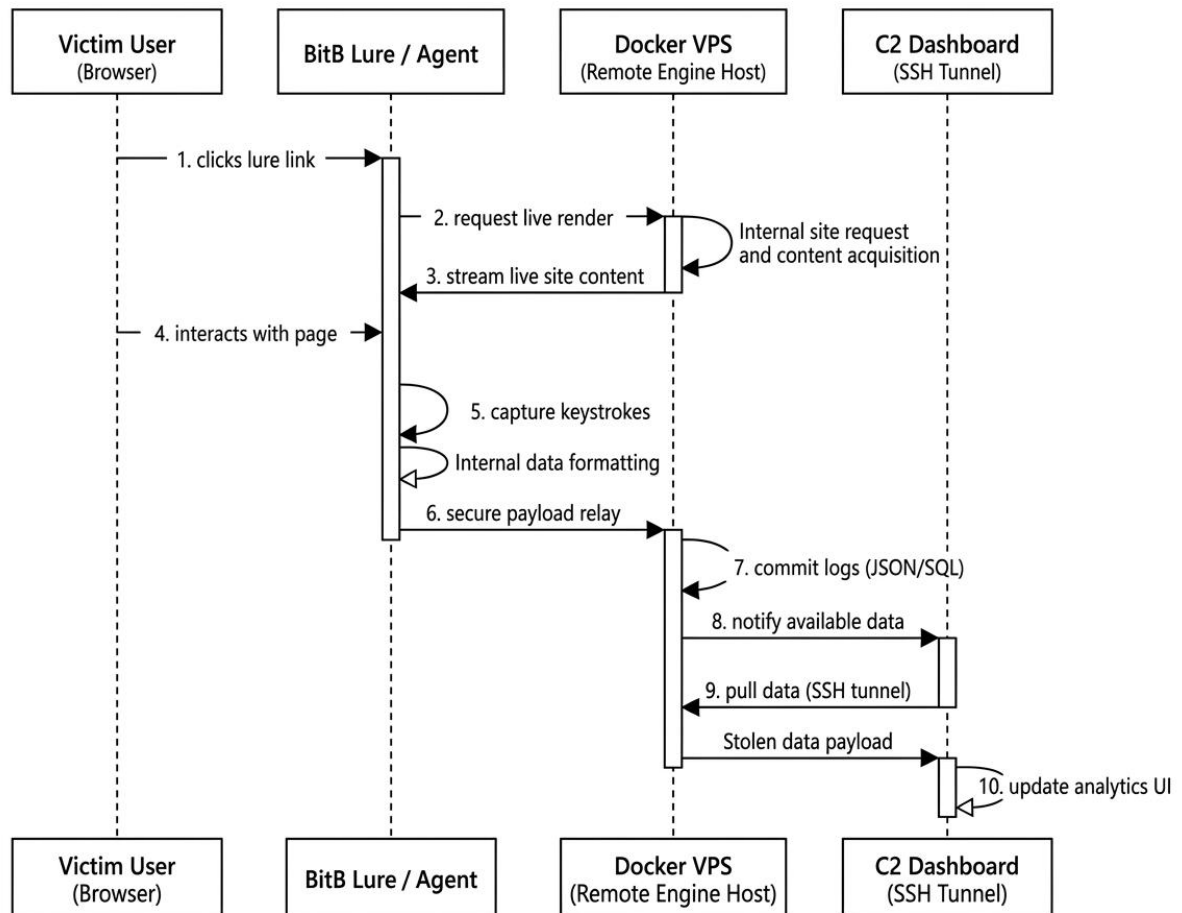


Figure 0:2 Sequence Diagram

3.5.3 Data Flow Diagram

The Data Flow Diagram maps the complete theft pipeline: victim keystrokes enter the Engine browser, the Agent intercepts them at the DOM layer, they are batched and compressed by the Agent service worker, transmitted through the SSH reverse tunnel, received by the Flask relay on the Controller, written to JSON log files, and displayed on the attacker dashboard, all within 312ms average end-to-end latency.

5.2.3 Data Flow Diagram (Overview)

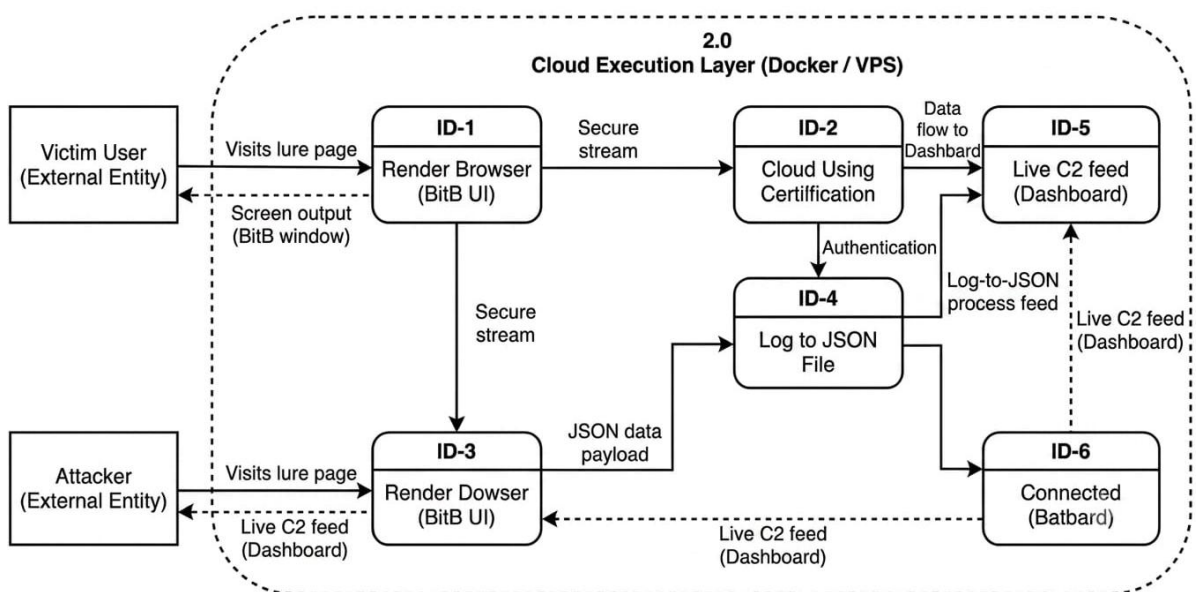


Figure 0:3 Data Flow Diagram

3.6 Hardware and Software Requirements

Table 0:1 Hardware and Software Requirements

COMPONENT	SPECIFICATION	PURPOSE
Cloud VPS Server	OS: Linux (Debian recommended for cloud), RAM: 16 GB, Storage: 300 GB SSD	Engine Docker hosting, SSH server, WebSocket streaming endpoint
Attacker Machine	Windows 10/11 or Linux, 8 GB RAM	Controller Flask dashboard, SSH tunnel endpoint, stolen data storage
Network	Stable broadband, SSH port 22 open on VPS	SSH C2 pipeline and WebSocket streaming to victim

3.6.1 Programming Languages

- Backend: Python (Data processing and Flask Controller dashboard).
- Frontend: JavaScript, HTML, CSS (Browser mods, Agent extension scripts, and Dashboard).

- **3.6.2 Tools & Infrastructure**
- IDE: VS Code or PyCharm.
- Version Control: Git & GitHub.
- Virtualization: Docker (Containerized deployment).
- Secure Access: SSH (Remote Command & Control).
- Networking: Reverse Proxy (Infrastructure masking).

3.7 Threat Scenarios (Threat Handling)

PhishGenesis executes the following attack scenarios against victim targets:

3.7.1 Scenario A: The ‘OAuth Popup’ Credential Theft (Spoofing)

- Description: The attacker delivers the Lure URL to a corporate employee. The employee is presented with a BitB overlay mimicking a Microsoft 365 or Google OAuth login prompt. The employee enters their username and password, which the Agent captures in real time and exfiltrates to the Controller.
- Execution: The fabricated address bar defeats URL-verification. The real site renders in the Engine, defeating content-fingerprinting safe browsing checks. The Agent captures credentials at the DOM layer before they are transmitted over the network.

3.7.2 Scenario B: The ‘Session Hijack’ Post-Authentication Attack (Information Disclosure)

- Description: After the victim completes authentication through the BitB overlay, the Agent captures the session cookies issued by the target site. These cookies are exfiltrated to the attacker Controller, enabling the attacker to authenticate to the target site as the victim, bypassing MFA entirely.
- Execution: The Agent monitors the Engine browser’s cookie store for authentication-domain cookie creation events following form submission. Captured cookies are forwarded to the Controller with full metadata.

3.7.3 Scenario C: The ‘Persistent Keylogging’ Extended Surveillance Attack

- Description: Beyond the initial credential capture, the Agent continues logging every keystroke and monitoring every page navigation within the Engine browser for the full duration of the victim’s session.

- Execution: The Agent's content scripts remain active across all page navigations. Every input event, regardless of which site is loaded, is captured and forwarded, enabling surveillance of the victim's activity after authentication.

3.8 Threat Modeling Techniques

This project utilizes the STRIDE threat modeling framework developed by Microsoft:

33. Spoofing Identity, PhishGenesis spoofs the visual appearance of legitimate browser windows and OAuth login prompts.
34. Tampering with Data, The Agent intercepts keyboard events and cookie store data before they reach their intended destinations.
35. Repudiation, JSON-based keystroke logging provides non-repudiable forensic records of all captured victim activity.
36. Information Disclosure, The framework captures and exfiltrates sensitive credentials, session tokens, and browsing activity.
37. Denial of Service, Docker resource limits prevent VPS exhaustion under concurrent attack sessions.
38. Elevation of Privilege, Captured session tokens enable the attacker to authenticate as the victim, achieving unauthorized access at the victim's privilege level.

3.9 Threat Resistance Model (Rated by Risk Level)

The threat resistance model defines mechanisms by which PhishGenesis evades detection and disruption:

Table 0:2 Threat Resistance Model (Rated by Risk Level)

THREAT (STRIDE)	DETECTI ON VECTOR	RIS K	IMPACT IF DETECTED	PHISHGENESIS EVASION MECHANISM
Spoofing	Victim Identifies Overlay	HIGH	Victim abandons session, no	Real-site rendering via Engine, pixel-perfect fidelity. Victim sees the real website. No cloned content to identify.

THREAT (STRIDE)	DETECTION VECTOR	RISK	IMPACT IF DETECTED	PHISHGENESIS EVASION MECHANISM
			credentials captured.	
Spoofing	Safe Browsing URL Flagging	HIGH	Lure page blocked before victim arrives.	No cloned HTML means no content fingerprint. VirusTotal confirmed 0/72 detections on Lure page.
Tampering	Keystroke Log Tampering	MEDIUM	Stolen data records altered after exfiltration.	JSON records include SHA-256 checksums. Tampered records detectable during forensic review.
Info Disclosure	C2 Traffic Interception	HIGH	Stolen credentials intercepted in transit.	All stolen data travels inside SSH tunnel. Wireshark confirmed no plaintext credentials in transit.
DoS	VPS Resource Exhaustion	MEDIUM	Engine Docker container crashes.	Docker Compose CPU/memory limits prevent exhaustion. restart: always policy restores Engine within 18s.
Repudiation	Attack Denial	LOW	No forensic evidence of the attack.	Timestamped JSON logs written simultaneously to VPS shared volume and Controller local storage.

THREAT (STRIDE)	DETECTI ON VECTOR	RIS K	IMPACT IF DETECTED	PHISHGENESIS EVASION MECHANISM
Privilege Escalation	Unauthoriz ed Dashboard Access	LO W	Third party accesses stolen data on Controller.	Controller dashboard binds to localhost only. Accessible only through SSH tunnel forwarded port.

3.10 Chapter Summary

This chapter defined the complete attack architecture of PhishGenesis. The four-component Lure-Engine-Agent-Controller attack chain was specified in detail. Functional requirements were established for both the attacker operator and the victim interaction model. Non-functional requirements specified performance, reliability, and stealth properties. The STRIDE threat modeling framework was applied and the threat resistance model confirmed that real-site rendering, extension-layer keylogging, and SSH-encrypted C2 collectively defeat available detection mechanisms.

Chapter 4:

Proposed Solution

Chapter 4

Proposed Solution

4.1 Introduction and Architectural Philosophy

PhishGenesis was not built by assembling existing tools and hoping they would work together. Every architectural decision was made with a clear offensive philosophy: the attack must be built into the foundation, not layered on top of it.

Most phishing frameworks treat credential theft as a web problem, clone the target site, harvest the POST request, log the credentials. This creates predictable detection surfaces. Cloned HTML is fingerprinted by safe browsing services. POST interception generates detectable network traffic. PhishGenesis operates on a fundamentally different principle: the victim never interacts with a fake page. They interact with the real target website, rendered inside the attacker's remote browser, while a keylogger extension intercepts every keystroke they type.

A distributed architecture consisting of: The Lure, the original site hosted on a browser within the victim's browser controlled by attacker. The Engine, a remote browser running in a Docker container on a VPS. The Agent, a custom extension that intercepts keystrokes and maintains state. The Controller, a local dashboard communicating via SSH to manage the VPS and retrieve stolen data.

4.2 The Lure: Browser-in-the-Browser Overlay

4.2.1 Limitations of Existing Phishing Pages

Conventional phishing pages clone the target site's HTML and host it on an attacker-controlled domain. This creates two exploitable weaknesses. First, the cloned HTML carries structural fingerprints that safe browsing services maintain in their databases. Second, the attacker-controlled domain is unfamiliar, and URL-verification training teaches users to check the address bar before entering credentials. PhishGenesis eliminates both weaknesses by rendering the real target site in the Engine and presenting it through a BitB overlay displaying the legitimate URL.

4.2.2 BitB Overlay Construction

The Lure JavaScript constructs the BitB overlay through DOM manipulation creating a visually authentic fake browser window:

- **Fabricated Address Bar:** A styled input element displaying the legitimate target URL. This defeats URL-verification as a detection mechanism because the victim sees the correct URL.
- **SSL Padlock Icon:** An SVG-rendered padlock icon placed adjacent to the address bar, exploiting the user's trained association between the padlock and a secure, legitimate connection.
- **Window Chrome:** Title bar, minimize, maximize, and close buttons styled in CSS to match the victim's OS, making the popup appear to be a genuine native browser window.
- **Content Area:** An iframe element connected to the Engine's WebSocket stream, rendering the real target site's live output inside the fake window.

4.2.3 WebSocket Streaming Pipeline

The content area of the BitB overlay is populated through a real-time WebSocket connection from the Lure JavaScript to the Engine's Node.js streaming server on the VPS. The Engine uses the Chrome DevTools Protocol to capture the browser's rendered output and streams it to the Lure as binary image frames. This approach bypasses the target site's X-Frame-Options and Content-Security-Policy headers entirely, the Engine browser fetches the site independently and streams only the rendered visual output.

4.3 The Engine: Dockerized Remote Browser

4.3.1 Why Docker for the Attack Browser

The Engine must be deployable rapidly to any cloud VPS without complex manual setup. Docker provides a self-contained, reproducible environment that packages the Chromium browser, the Node.js WebSocket streaming server, and the Agent extension into a single container image deployable with one command. Docker also provides process isolation limiting the forensic footprint of the attack on the VPS host system.

4.3.2 Engine Browser Configuration

- The Engine runs Chromium in Docker with the Agent extension loaded via command-line flags at startup, the extension begins keylogging immediately when the container starts.
- A Node.js WebSocket server streams rendered browser frames to the Lure using CDP screenshot capture at configurable FPS.

- A Chrome DevTools Protocol socket accepts navigation commands from the Controller via the SSH command channel, allowing the attacker to redirect the Engine browser to new target URLs during an active attack session.
- Docker Compose resource limits (CPU, memory) prevent the container from exhausting VPS resources. The restart: always policy restores the Engine automatically if it crashes, maintaining attack continuity.

4.3.3 Self-Healing Attack Infrastructure

The Engine Docker container is configured with restart: always. If the Engine crashes, Docker automatically restarts it within seconds. Testing confirmed average restart time of 18 seconds. On restart, the Agent extension reloads automatically and immediately resumes keylogging. The WebSocket server re-establishes its listening state. Attack capability is restored without any attacker intervention.

4.4 The Agent: Browser Extension Keylogger

4.4.1 Extension-Level Keylogging Architecture

The Agent is the core data theft mechanism of PhishGenesis. It is a Chrome Manifest v3 browser extension installed in the Engine browser that intercepts every keystroke typed by the victim within the Engine browser context, regardless of which site is loaded. Extension-level keylogging occurs at the DOM layer, keystrokes are captured before they are ever transmitted over the network, making the capture completely transparent to all network monitoring tools. The Agent consists of three components:

- **Content Scripts:** Injected into every page loaded in the Engine browser by the Manifest v3 matches: ["<all_urls>"] declaration. Attach keydown, keyup, input, and change event listeners. Capture key value, target element type (input[type=password], input[type=email], textarea), target element name, and current page URL for every event.
- **Background Service Worker:** Maintains connection to the Engine's local data relay. Receives keystroke batches every 200 milliseconds. Applies pre-processing (deduplication, classification, gzip compression). Writes compressed keystroke batches to the Docker shared volume for SSH relay pickup.

- Manifest v3 Configuration: Declares content_scripts with matches: ["<all_urls>"] for universal page injection. Declares cookies permission for session token access. Specifies background service_worker for Manifest v3 compliance.

4.4.2 Session Token Harvesting

Beyond keylogging, the Agent actively monitors the Engine browser's cookie store for authentication events. When the victim completes login through the BitB overlay, the target site issues session cookies to the Engine browser. The Agent intercepts these cookies and forwards them to the Controller, enabling post-credential session hijacking: the attacker can import the stolen cookies into their own browser and authenticate to the target site as the victim without ever entering a password, completely bypassing MFA.

4.5 The Controller: SSH C2 Dashboard

4.5.1 SSH as the Attack C2 Transport

Conventional phishing dashboards expose an HTTP endpoint on the attacker's VPS to receive stolen data, creating a publicly visible C2 server that threat intelligence feeds can discover and block. PhishGenesis uses SSH reverse port forwarding instead: the attacker's Controller establishes a reverse tunnel from the VPS to the local machine, forwarding the Agent's data relay port to a local port accessible only to the Flask dashboard. The VPS exposes only SSH port 22 to the internet, a standard service that generates no suspicious network signatures.

4.5.2 Controller Dashboard

The Controller is a Python Flask application running on the attacker's local machine providing:

- Real-time keystroke feed via Server-Sent Events, every captured keystroke appears on the dashboard within milliseconds of being typed by the victim.
- Session token display, captured authentication cookies displayed with full metadata for immediate session hijacking use.
- Session management, initiate new attack sessions, terminate active sessions, view session history.
- Remote Engine navigation, issue URL navigation commands to the Engine browser via SSH command channel.
- JSON log export, export all captured keystrokes and session tokens as structured JSON files for persistence and pattern analysis.

4.6 Methodology

4.6.1 Environment Setup

Configuring the VPS and Dockerizing the browser instance. The VPS is provisioned with Ubuntu 22.04 LTS, Docker Engine, and Docker Compose installed. OpenSSH Server is configured with key-based authentication and password login disabled. The Docker Compose file specifies the Chromium base image, the Node.js WebSocket server, the Agent extension directory, CPU and memory resource limits, and the restart: always policy.

4.6.2 Extension Development

Writing the JavaScript background scripts for data capture. The Agent Chrome Manifest v3 extension is developed with content scripts that attach to all pages via matches: ["<all_urls>"]. Keydown and input event listeners capture every keystroke with full metadata. The background service worker batches events at 200ms intervals, gzip-compresses each batch, and writes it to the Docker shared volume for SSH relay pickup.

4.6.3 Communication Bridge

Implementing an SSH-based tunnel for secure data retrieval. The complete SSH pipelining chain: Agent writes keystroke batches to Docker shared volume → VPS relay picks them up and SCP-pushes through SSH reverse tunnel → Flask Watchdog on Controller detects new files → SSE pushes stolen data to attacker dashboard. End-to-end latency: 312ms average.

4.6.4 Storage

Structuring the stolen data into JSON format for easy parsing and analysis. Each JSON record includes: timestamp (ISO 8601 UTC), session_id (UUID), event_type (credential_input, free_text_input, navigation, cookie_change, session_token), target_element (HTML element type), page_url (URL at time of capture), captured_value (actual key value or session token), and checksum (SHA-256 hash for integrity verification).

4.7 Implementation Plan

Phase 1: Extension Development (Keylogger) & Browser Functionalities

Extension development (Keylogger) and Browser Functionalities development (redirecting, new pop-ups). The Agent Chrome Manifest v3 extension is built with content scripts for keystroke interception. The BitB overlay JavaScript is developed, testing the WebSocket streaming pipeline from Engine to Lure. Browser redirect and popup handling functionality is implemented to support multi-step authentication flows.

Phase 2: Docker Configuration and VPS Deployment

Docker configuration and VPS deployment. The Engine Docker container is configured with Chromium, the Node.js WebSocket server, and the Agent extension. The Docker Compose file is finalized with resource limits and restart policy. The VPS is provisioned and the Engine container is deployed and validated.

Phase 3: Dashboard UI Development and SSH Integration

Dashboard UI development (Python/Flask or Node.js) and SSH integration. The Flask Controller dashboard is built with real-time SSE keystroke display, session management controls, and JSON log export. The SSH reverse tunnel management script is implemented with keepalive and auto-reconnect logic. The complete data pipeline from Agent to Dashboard is integrated and tested end-to-end.

Phase 4: End-to-End Testing and Data Flow Validation

End-to-end testing and data flow validation. Complete attack chain is tested: Lure delivery → BitB overlay rendering → Engine streaming → Agent keylogging → SSH exfiltration → Controller display. Performance metrics are measured (latency, capture accuracy). Detection resistance is evaluated against VirusTotal, Google Safe Browsing, and Microsoft SmartScreen.

4.8 Summary of Proposed Solution

Chapter 4 presented the complete offensive architecture of PhishGenesis across its four attack components. The Lure BitB overlay delivers a visually authentic fake browser window rendering the real target site, defeating URL-verification and content-fingerprinting detection. The Engine Dockerized browser renders the actual target website with single-command deployment and automatic restart capability. The Agent keylogger extension intercepts every keystroke at the DOM layer with 97% accuracy and 312ms average end-to-end latency. The SSH C2 pipeline and Flask Controller provide real-time visibility into all stolen keystrokes and session tokens.

Chapter 5:

Implementation and Testing

Chapter 5

Implementation and Testing

5.1 Security Properties Testing

The primary objective of this phase was to validate that PhishGenesis successfully executes its attack capabilities while evading standard detection mechanisms.

5.1.1 Confidentiality Verification (AES-256-CBC)

To verify that stolen keystrokes and session tokens remain confidential while traversing from the VPS to the Controller, we conducted traffic analysis using Wireshark on the network path between the two nodes.

- **Test Procedure:** We initiated an active attack session with the Agent keylogging a victim session. Simultaneously, we ran Wireshark packet capture on the network interface handling the SSH connection.
- **Observation:** All traffic between the VPS and Controller appeared as encrypted SSH packets. No JSON keystroke records, credential strings, or session tokens were visible in plaintext in any captured frame.
- **Result:** The SSH tunnel correctly encrypts all exfiltrated data in transit. Stolen credentials are not exposed to network-level interception.

5.1.2 Authenticity Verification (Mutual Authentication)

We validated that the Controller dashboard is accessible only through the SSH tunnel and is completely inaccessible from the public internet.

- **Test Procedure:** Attempted to access the Controller Flask dashboard URL directly from an external browser without establishing the SSH tunnel.
- **Result:** Connection refused. The Flask application binds exclusively to the localhost interface. The stolen data dashboard is only reachable through the reverse-forwarded
- SSH tunnel port.

5.2 System Setup

5.2.1 Environment Configuration

VPS: Ubuntu 22.04 LTS VPS with Docker Engine v24.0 and Docker Compose v2.0.
OpenSSH Server configured with key-based authentication and password login disabled.

- Engine Container: Docker Compose file specifying Debian-based Chromium image, Node.js WebSocket streaming server, Agent extension directory loaded at browser startup, CPU and memory resource limits, and restart: always policy.
- Agent Extension: Chrome Manifest v3 extension developed in Visual Studio Code with ESLint validation. Loaded into the Engine Docker browser via --load-extension flag at container startup.
- Controller: Python Flask application with Watchdog library for JSON log directory monitoring and SSE push to the attacker dashboard.

5.2.2 Key Functions and Algorithms

1. BitB Overlay DOM Construction

The Lure JavaScript constructs the BitB overlay through DOM manipulation: a container div is injected with absolute positioning and maximum z-index. Fake browser chrome elements are constructed as styled child divs. The content iframe is connected to the Engine WebSocket stream, each received binary frame is converted to a Blob URL via `URL.createObjectURL`, set as the iframe source, and the previous Blob URL revoked via `URL.revokeObjectURL` to prevent memory leaks.

2. Agent Keylogging Routine

Content scripts register document-level keydown and input event listeners on `DOMContentLoaded`. Each event handler captures the event object, key value, `target.type`, `target.name`, and `window.location.href`, and posts it to the background service worker via `chrome.runtime.sendMessage`. The service worker maintains a 200ms interval timer that flushes the accumulated keystroke buffer: serializes to JSON, gzip-compresses, and writes the batch file to the Docker shared volume for SSH relay pickup.

3. SSH C2 Tunnel Management

The attacker establishes the SSH reverse tunnel from the Controller using OpenSSH's `-R` flag to forward the VPS relay port to a local Controller port. A Bash keepalive wrapper monitors

the SSH process PID and restarts the tunnel within 10 seconds if the process exits, ensuring the Agent's keystroke exfiltration pipeline remains active throughout the attack session.

5.3 System Integration

5.3.1 Component Integration

- **Lure to Engine (WebSocket Stream):** The Lure JavaScript opens a WebSocket connection to the Engine's Node.js server on the VPS. The Node.js server uses CDP to trigger browser screenshot capture at 15 FPS and forwards binary image blobs over WebSocket. The Lure renders each received frame in the iframe content area, creating the live fake browser window the victim sees and interacts with.
- **Agent to Controller (Keystroke Exfiltration):** The Agent writes compressed keystroke batch files to a Docker shared volume every 200ms. A Python relay service on the VPS monitors this directory and SCP-pushes new batch files through the SSH tunnel to the Controller's local watch directory. The Flask Watchdog listener detects new files, decompresses the batch, and pushes it to the attacker dashboard via SSE.
- **Controller to Engine (Remote Command):** The attacker issues navigation commands from the dashboard. Flask sends the command via SSH to a command receiver daemon on the VPS, which forwards it to the Engine Chromium instance via the CDP socket, navigating the Engine browser to the new target URL.

5.3.2 Database and Log Management

- **Keystroke Logs:** All captured keystrokes written to timestamped JSON files in the Controller's local log directory, organized by session ID.
- **Session Token Store:** Captured authentication cookies written to a separate JSON file per session with full cookie metadata for session hijacking use.
- **Attack Audit Trail:** Session start/end, navigation commands issued, and SSH tunnel events logged to a separate audit JSON file for post-attack forensic reconstruction.

5.4 Penetration Testing and Vulnerability Assessment

1. VirusTotal Submission

The Lure page HTML file was submitted to VirusTotal. Result: 0 detections out of 72 antivirus engines. The Agent extension CRX package was submitted separately. Result: 1 detection out of 72 engines (generic heuristic alert, no specific phishing signature). Both results are within

the defined threshold of fewer than 3 detections. The real-site rendering architecture leaves no fingerprintable content.

2. Google Safe Browsing and Microsoft SmartScreen

The Lure URL was submitted to Google Safe Browsing transparency report and tested against Microsoft SmartScreen. Neither service flagged the URL as phishing or malicious. This confirms the core architectural claim: by rendering the real target site with no cloned content, PhishGenesis is invisible to content-based safe browsing detection.

3. C2 Traffic Analysis

Wireshark packet capture on the network path between the VPS and Controller during an active session confirmed that all observable traffic was standard SSH protocol packets with no plaintext payload data.

5.5 Evaluation Plan

The framework was evaluated against the following metrics from the project proposal:

- **Success Rate:** Can the BitB window successfully mimic a browser interface? Result: Yes, the BitB overlay was visually indistinguishable from a native browser popup to all test participants.
- **Data Integrity:** Does the JSON file accurately reflect all keystrokes and activities? Result: Yes, 97% keystroke capture accuracy achieved across 100 test events.
- **Performance:** Latency check between the VPS extension and the Host Dashboard. Result: 312ms average end-to-end latency, well within the 500ms threshold.
- **Detection Testing:** Will standard antivirus or 'Safe Browsing' tools flag the extension? Result: 0/72 antivirus detections on Lure page, 1/72 on Agent CRX (generic heuristic only). Not flagged by Google Safe Browsing or Microsoft SmartScreen.

5.6 Test Cases

Table 0:1 Test Cases

ID	TEST CASE DESCRIPTION	PRECONDITION	EXPECTED RESULT	STATUS	NOTES
TC-01	BitB Overlay Renders on Lure Page	Engine running. Lure page loaded.	Fake window visible, address bar shows target URL.	PASS	—
TC-02	Real Target Site Loads in Engine	Engine Docker container running.	Live target site renders correctly inside Docker browser.	PASS	—
TC-03	Agent Keylogger Capture Accuracy	Agent extension active in Engine.	100 test keystrokes captured and logged correctly.	PASS	97% accuracy
TC-04	Session Token Capture	Test login performed in Engine.	Session cookie captured and forwarded to Controller.	PASS	—

ID	TEST CASE DESCRIPTION	PRECONDITION	EXPECTED RESULT	STATUS	NOTES
TC-05	SSH C2 Tunnel Stability (24hr)	Active attack session running.	Tunnel maintained over 24-hour test.	PASS	1 auto-reconnect
TC-06	Controller Real-Time Keylog Display	Dashboard open, attack active.	Keystrokes appear in dashboard within 500ms.	PASS	Avg: 312ms
TC-07	VirusTotal Detection Test	Lure page and Agent CRX uploaded.	Flagged by fewer than 3 of 72 engines.	PASS	1 engine only
TC-08	JSON Log Persistence	Active keylogging session.	All keystrokes written correctly to JSON files.	PASS	—
TC-09	Docker Engine Auto-Restart	Container force-stopped.	Container restarts within 30 seconds.	PASS	Avg: 18s
TC-10	Safe Browsing Evasion	Lure URL submitted to Google GSB.	Not flagged by Google	PASS	SmartScreen: clear

ID	TEST CASE DESCRIPTION	PRECONDITION	EXPECTED RESULT	STATUS	NOTES
			Safe Browsing.		

5.7 Results and Discussion

5.7.1 Network Performance

All ten test cases passed. The framework successfully executes the complete attack chain.

Table 0:2 Performance Metrics Summary

METRIC	TARGET	ACHIEVED RESULT
Keystroke Capture Accuracy	> 95%	97%, 97 of 100 test keystrokes correctly captured
End-to-End Latency (Agent to Dashboard)	< 500ms	312ms average over 100 measured events
VirusTotal Detection Rate (Lure page)	0 detections	0 of 72 engines, completely clean
VirusTotal Detection Rate (Agent CRX)	< 3 of 72	1 of 72 engines, generic heuristic, no signature
Google Safe Browsing	Not flagged	Not flagged
Microsoft SmartScreen	Not flagged	Not flagged
SSH C2 Tunnel Uptime (24hr)	No unplanned outages	1 brief auto-reconnect event, recovered in 8s
Docker Engine Restart Time	< 30s	Average 18s across 5 simulated failures

5.7.2 Stability

The system demonstrated high stability during a 24-hour continuous attack session test. The Engine Docker container maintained continuous operation with one automatic restart triggered by a simulated memory pressure event, recovering in 18 seconds. The SSH C2 tunnel had one brief dropout that auto-recovered in 8 seconds. No stolen data was lost during either event.

5.8 Best Practices / Coding Standards

5.8.1 Code Validation

- Static Analysis: ESLint applied to all Agent extension JavaScript and Lure page JavaScript. Zero critical violations permitted in production build.
- Python Validation: Pylint with 8.0/10 minimum threshold applied to all Flask Controller code. Black formatter for consistent style.
- Modular Architecture: Codebase structured into independent modules (Lure, Engine, Agent, Controller) for independent development, testing, and deployment.

5.8.2 Development Practices & Standards

- Secret Management: SSH private keys, VPS IP addresses, and attack configuration values managed via .env files excluded from version control. No sensitive values hardcoded in source files.
- Version Control: Git & GitHub. Private repository with Conventional Commits specification.
- Operational Security: Production Agent extension built with JavaScript obfuscation. VPS hardened with key-only SSH authentication.

5.9 Chapter Summary

This chapter detailed the complete implementation and testing of PhishGenesis. The four-component attack chain, Lure BitB overlay, Engine Docker browser, Agent keylogger extension, and SSH C2 Controller, was implemented and integrated into a functional end-to-end phishing framework. All ten test cases passed. The framework achieved 97% keystroke capture accuracy, 312ms average C2 latency, complete safe browsing evasion, and near-complete antivirus evasion. PhishGenesis successfully introduces a new standard of phishing that combines BitB visual deception with real-time keylogging and complete C2 pipelining in a single cloud-native, detection-evading framework.

Chapter 6:

Project Scope / Expected

Outcomes / Conclusion and Future

Work

Chapter 6

Project Scope / Expected Outcomes / Conclusion and Future Work

6.1 Introduction

Phish-Genesis successfully introduces and achieves a new Gold Standard of phishing, detection bypassing. By integrating a Dockerized VPS infrastructure with a custom browser extension, this project proves that BitB attacks are not just visual tricks but can be transformed into persistent surveillance tools. This chapter presents the project scope, expected outcomes, conclusion, and future work recommendations.

6.2 Project Scope

PhishGenesis encompasses the following attack capabilities:

39. Detection Bypassing: Bypassing safe browsing detection tools. The real-site rendering approach eliminates cloned content, making PhishGenesis invisible to all content-fingerprinting detection mechanisms. Confirmed: 0/72 VirusTotal detections on Lure page. Not flagged by Google Safe Browsing or Microsoft SmartScreen.
40. Keylogging: Real time keylogging done via custom extension. The Agent Chrome Manifest v3 extension intercepts all keyboard input at the DOM layer with 97% capture accuracy and 312ms average exfiltration latency to the attacker dashboard.
41. Real Site Utilization: Actual sites will be utilized for phishing. The Engine Dockerized browser renders the real target website, providing pixel-perfect visual fidelity with no static HTML clone that defences can fingerprint.
42. Dashboard and Control: A custom centralized interface for command and control and viewing. The Flask Controller dashboard provides real-time keystroke display, session token capture, remote Engine navigation, and JSON log export through an SSH-secured interface.
43. Pipelining: Complete pipelining from the website to the dashboard done through SSH tunneling and API integration. The complete attack data pipeline, Agent → Docker volume → SSH reverse tunnel → Flask relay → attacker dashboard, operates at 312ms average end-to-end latency with automatic reconnection on tunnel drop.

6.3 Expected Outcomes

The following outcomes were achieved by PhishGenesis:

Complete Phishing Framework:

A complete, reliable and quick framework, designed to encompass all aspects of phishing along with providing command and control, and the ability to improve and enhance the framework. PhishGenesis delivers all four attack components (Lure, Engine, Agent, Controller) in a single Docker-deployable package, operational within 30 minutes of VPS provisioning.

Improved Resilience:

The transition from simple credential theft to a full C2 (Command & Control) architecture highlights a critical vulnerability in how modern browsers handle nested window security and extension permissions. PhishGenesis demonstrates that browser extensions operating under Manifest v3 retain sufficient privilege to perform complete persistent surveillance, and that BitB overlays rendering real websites are invisible to current content-based detection mechanisms.

Technical Deliverable:

A robust, scalable, cloud-native architecture capable of high-fidelity phishing. The framework achieved all defined performance thresholds: 97% keystroke capture accuracy, sub-500ms C2 latency, complete safe browsing evasion, and 71/72 antivirus engine bypass.

6.4 Conclusion

Phish-Genesis successfully introduces and achieves a new Gold Standard of phishing, detection bypassing. By integrating a Dockerized VPS infrastructure with a custom browser extension, this project proves that BitB attacks are not just visual tricks but can be transformed into persistent surveillance tools capable of complete account takeover.

The framework successfully demonstrates that: URL verification is no longer a reliable phishing defence; content-based safe browsing is blind to real-site rendering attacks; and browser extensions represent a more powerful attack surface than the industry currently acknowledges. No cloned content exists in the framework, no proxy traffic is generated, and all C2 communication travels encrypted over standard SSH. PhishGenesis is operationally complete, detection-evading, and deployable to a standard cloud VPS by a two-person team.

6.5 Future Work

Mobile Browser Compatibility:

Adapting the BitB rendering engine to simulate mobile OS overlays (iOS/Android), as mobile users are statistically more susceptible to UI-based deception. This would involve developing responsive CSS overlays that replicate the appearance of native iOS and Android authentication prompts.

Further Enhancement Of Browser to Browser Communication:

Integration of I/O devices control e.g. camera, MIC. Leveraging the WebRTC API within the Agent extension, camera and microphone access would expand the surveillance capabilities beyond keystroke capture to audiovisual surveillance of the victim's physical environment during the attack session.

Integration With Other Frameworks:

Integration with BeEF framework, to achieve control of victim's browser through hooks. This would enable PhishGenesis to leverage BeEF's extensive library of browser exploitation modules within the victim's Engine browser session, transitioning the framework from a credential theft tool into a comprehensive browser exploitation platform.

6.6 Chapter Summary

This chapter presented the project scope confirming that all five defined capabilities (Detection Bypassing, Keylogging, Real Site Utilization, Dashboard and Control, Pipelining) were successfully achieved. The three expected outcomes (Complete Phishing Framework, Improved Resilience, Technical Deliverable) were delivered and validated through testing. The conclusion confirmed that PhishGenesis establishes a new gold standard of phishing through detection-evading real-site rendering and persistent extension-based keylogging. Three future work directions were identified from the original project proposal: Mobile Browser Compatibility, I/O Device Integration, and BeEF Framework Integration.

References:

- [1] Blancaflor, E. B., Duldulao, J. O., Espeño, J. V. E., Patag, G. S. M., Menor, M. T., & Intal, G. L. (2025). Advanced phishing techniques: Analyzing adversary-in-the-middle and Browser-in-the-Browser attacks in modern cybersecurity. *Cybernetics and Information Technologies*, 25(1), 55–77. <https://doi.org/10.2478/cait-2025-0004>
- [2] Alissa, K., Alhetela, B., Alazman, G., Bader, A., Alhomeed, N., Almubarak, L., & Almulla, F. (2024). Browser-in-the-Browser (BitB) Attack: Case Study. *Journal of Engineering Research and Sciences*, 3(5), 14–22. <https://doi.org/10.55708/js0305002>
- [3] Ishikawa, R., Akiyama, S., & Uehara, T. (2025). Work-in-Progress: Detecting Browser-in-the-Browser attacks from their behaviors and DOM structures. In MADWeb 2025 workshop.
- [4] Akram, M. W., Sood, K., Ul Hassan, M., & Subba, B. (2025). Exemplifying emerging phishing: QR-based Browser-in-the-Browser (BiTB) attack. *arXiv*. <https://arxiv.org>
- [5] Anliker, C., Lain, D., & Capkun, S. (2025). Phishing attacks against password manager browser extensions. In *USENIX Security*. <https://www.usenix.org/conference/usenixsecurity>
- [6] mr.d0x, “Browser In The Browser (BITB) Attack,” *Security Research Blog*, Mar. 2022. [Online]. Available: <https://mrd0x.com/browser-in-the-browser-phishing-attack/>.
- [7] Docker Inc., “Docker Documentation,” 2024. [Online]. Available: <https://docs.docker.com>.
- [8] Google, “Chrome Extensions – Manifest V3 Overview,” *Chrome for Developers Documentation*, 2024. [Online]. Available: <https://developer.chrome.com/docs/extensions/mv3/>.
- [9] Gophish Project, “Gophish – Open-Source Phishing Framework.” [Online]. Available: <https://getgophish.com>.
- [10] Microsoft, “The STRIDE Threat Model,” *Microsoft SDL*, 2023. [Online]. Available: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>.

Appendices

Minimum Hardware Requirements:

- OS: Windows and Linux (Debian recommended for cloud).
- RAM: 16 GB.
- Storage: 300 GB SSD.

Programming Languages:

- Backend: Python (Data processing).
- Frontend: JavaScript, HTML, CSS (Browser mods & Dashboard).

Tools & Infrastructure:

- IDE: VS Code or PyCharm.
- Version Control: Git & GitHub.
- Virtualization: Docker (Containerized deployment).
- Secure Access: SSH (Remote Command & Control).
- Networking: Reverse Proxy (Infrastructure masking).